AI Assisted Coding-15.4

Name: B.Akshara

Ht.no:2303A51279

Batch.no:05

Task 1– Real-Time Application: Online Food Ordering API

Prompt:

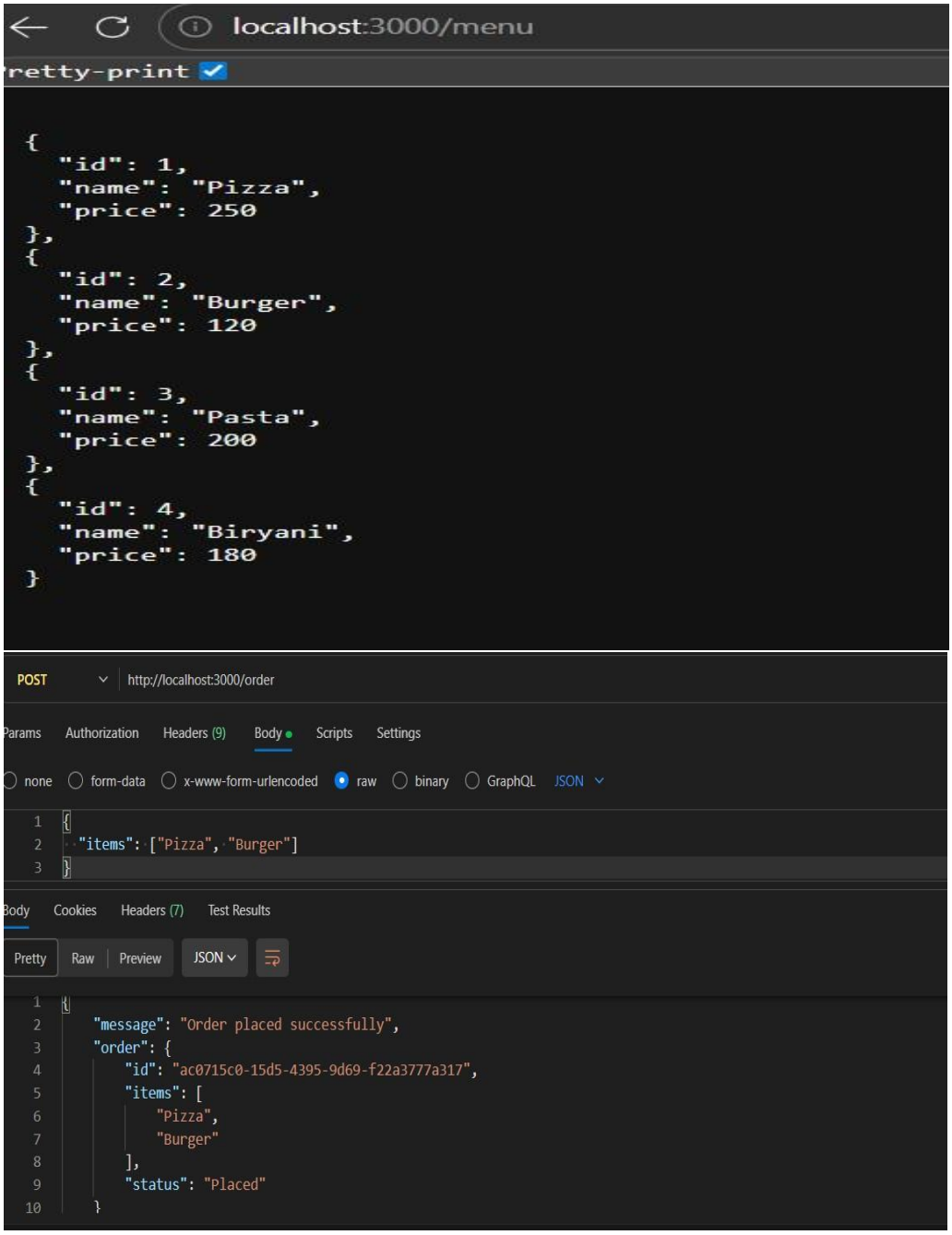
Create a simple REST API for an Online Food Ordering System using Node.js and Express.

Include endpoints:

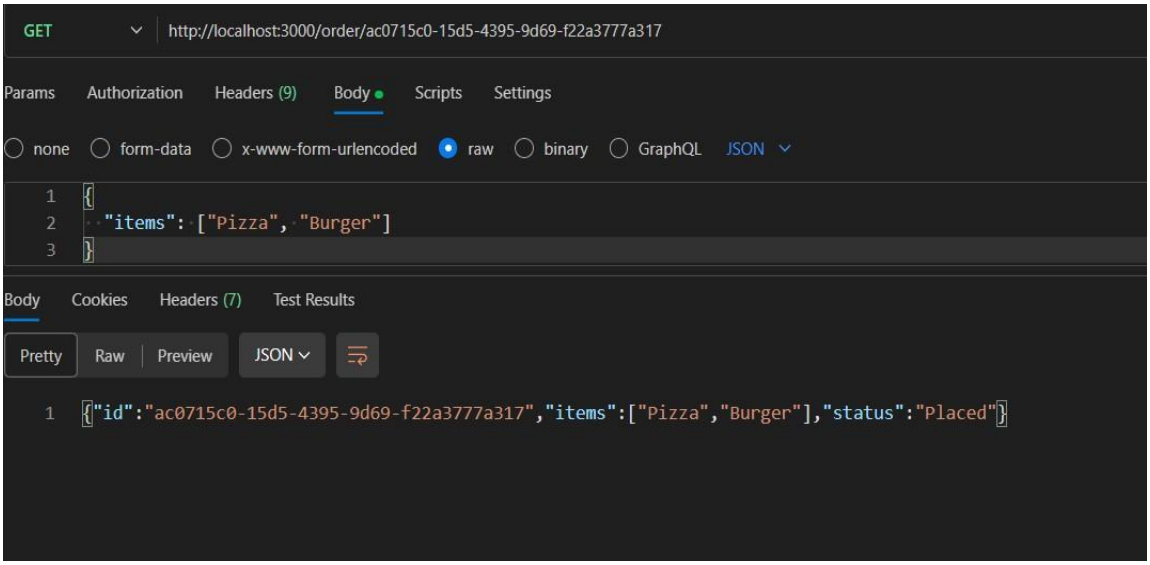
- GET /menu → list food items
- POST /order → create order
- GET /order/:id → get order status • PUT /order/:id → update order
- DELETE /order/:id → cancel order

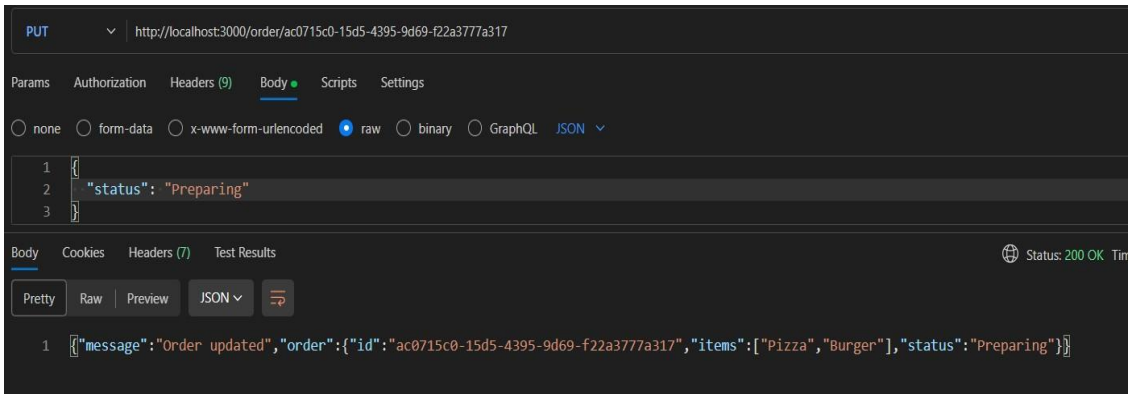
Use in-memory storage, UUID for IDs, proper status codes, and return clean JSON responses. Provide complete working code in one file (server.js) with run instructions. Also suggest improvements like authentication and pagination.

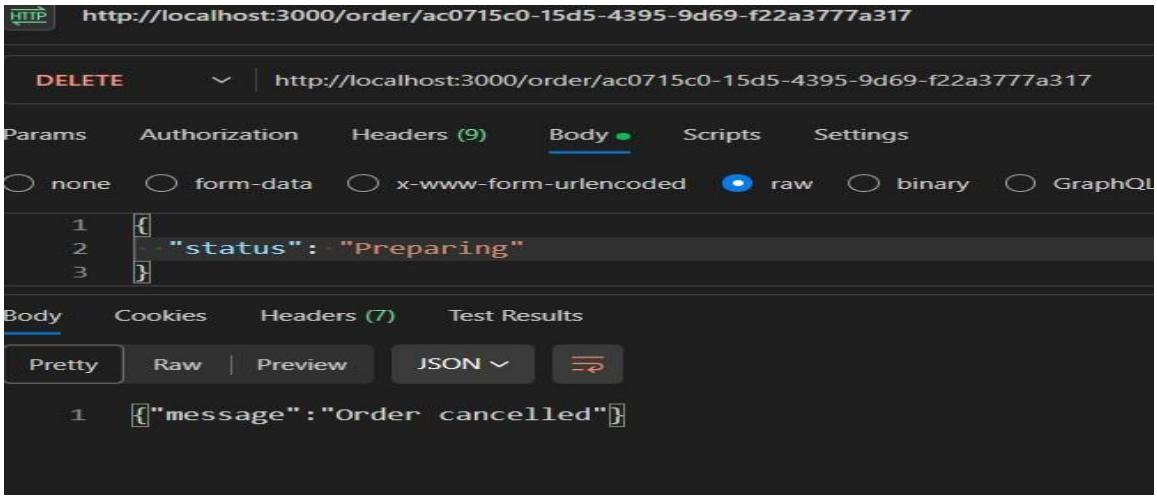
Output :


localhost:3000/menu
retty-print
{
"id": 1,
"name": "Pizza",
"price": 250
},
{
"id": 2,
"name": "Burger",
"price": 120
},
{
"id": 3,
"name": "Pasta",
"price": 200
},
{
"id": 4,
"name": "Biryani",
"price": 180
}

POST
http://localhost:3000/order
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"items": ["Pizza", "Burger"]
}
Body
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
JSON
{
"message": "Order placed successfully",
"order": {
"id": "ac0715c0-15d5-4395-9d69-f22a3777a317",
"items": [
"Pizza",
"Burger"
],
"status": "Placed"
}







Explanation : This code implements a simple API for an online food ordering system using Flask. It allows users to view the menu, place orders, track orders, update existing orders, and cancel orders. The menu and orders are stored in dictionaries, and the API handles various HTTP methods to perform the necessary operations on the orders.

Task 2 – Employee Payroll API

Prompt:

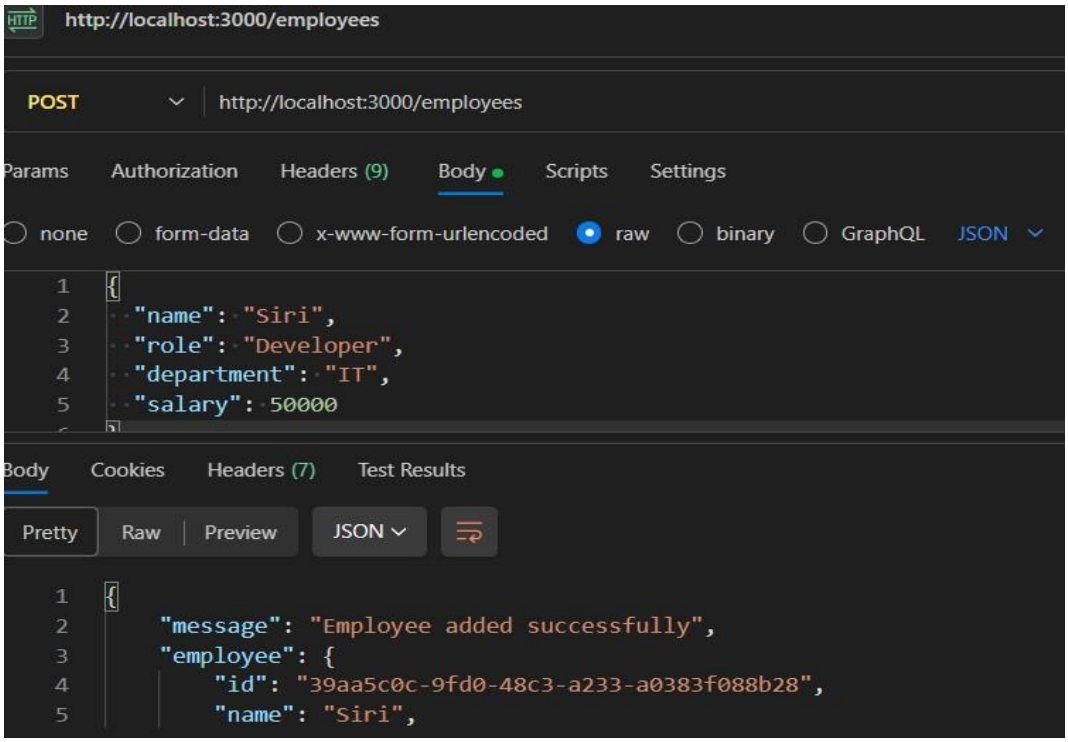
Create a REST API for an Employee Payroll System using Node.js and Express.

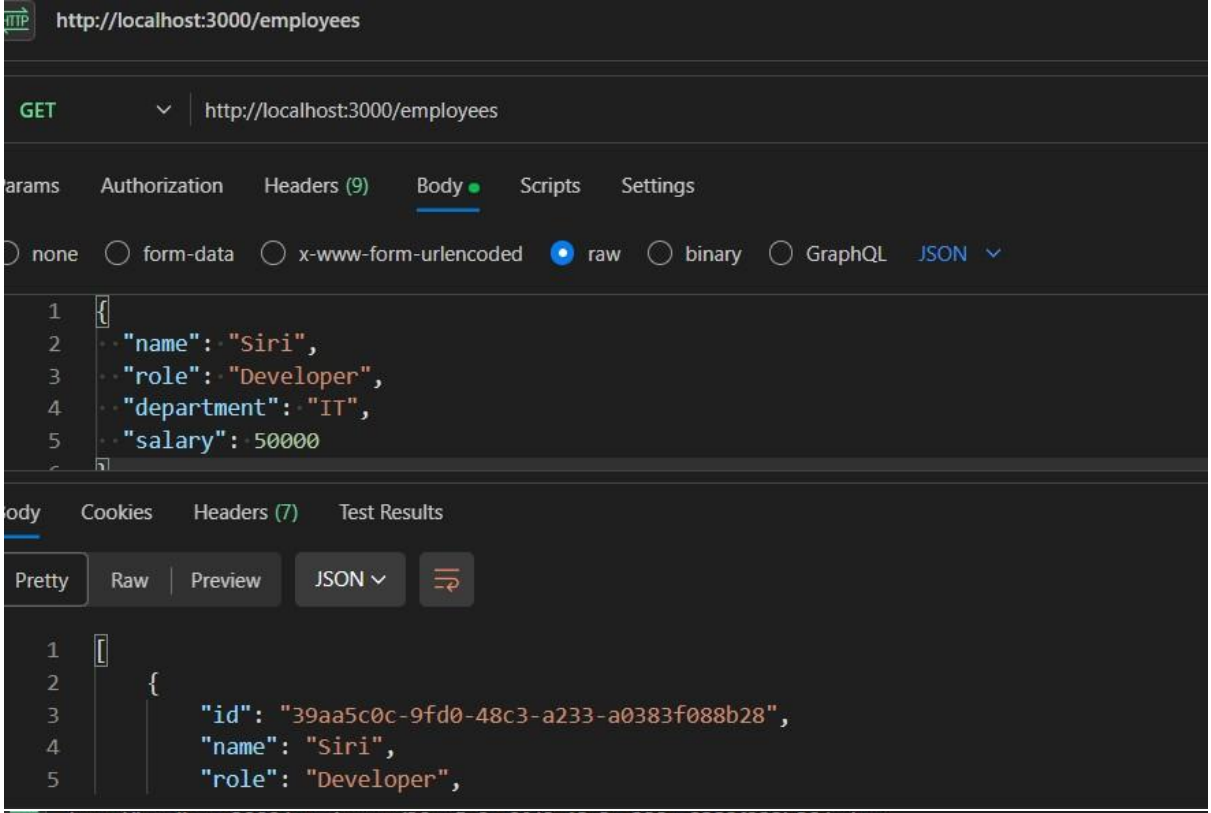
Include endpoints:

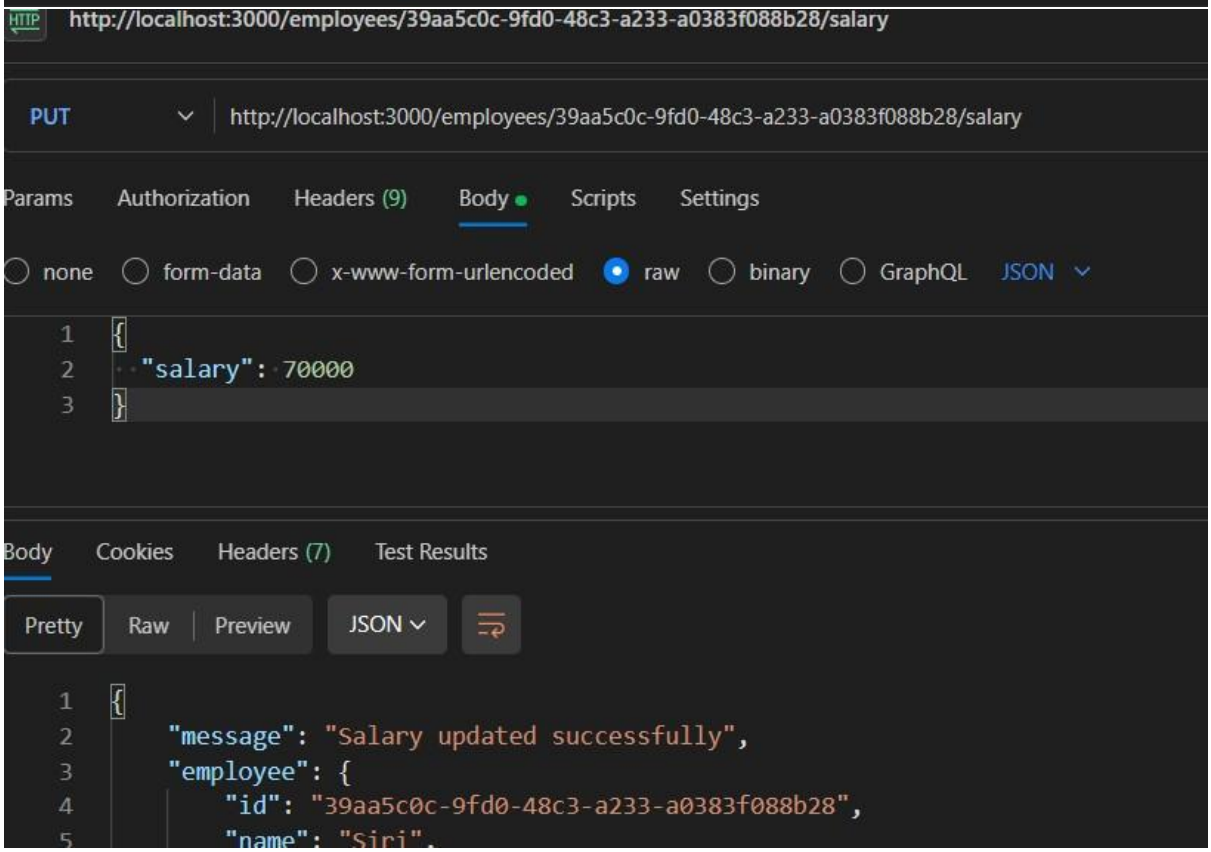
- GET /employees → list all employees
- POST /employees → add employee with salary details
- PUT /employees/:id/salary → update employee salary
- DELETE /employees/:id → remove employee

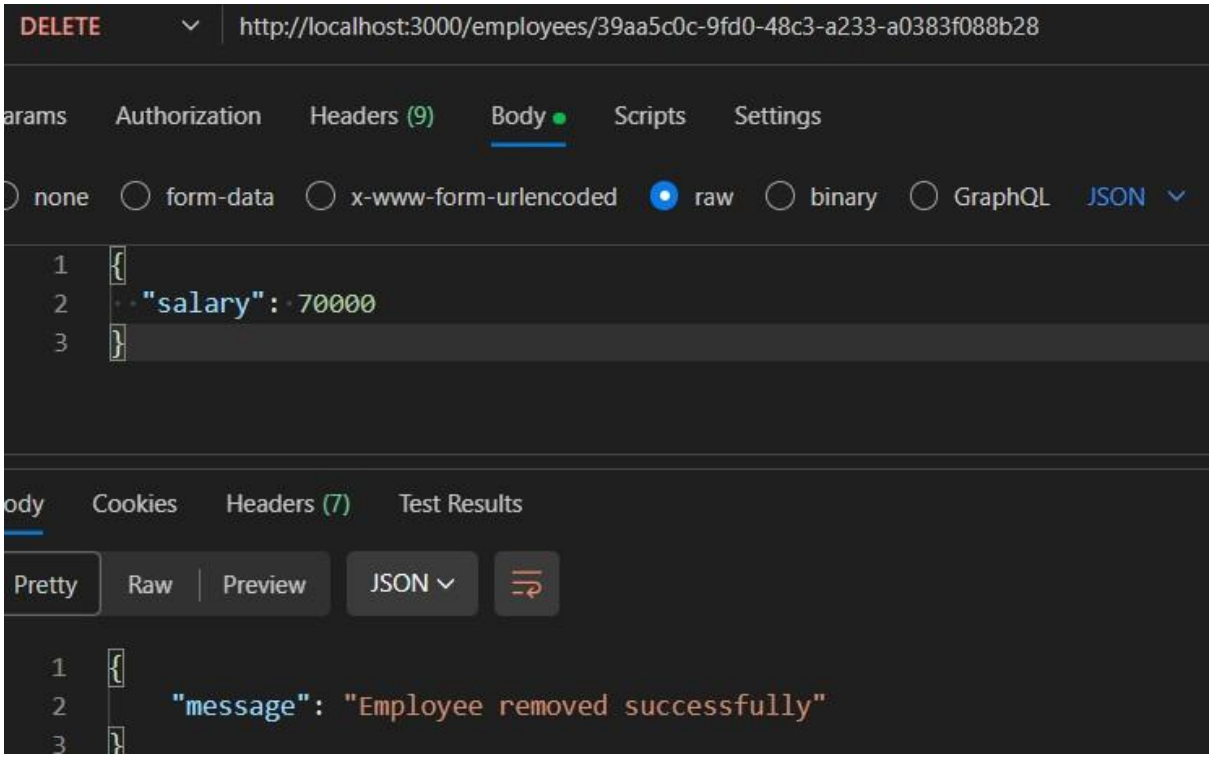
Use in-memory storage, unique IDs, input validation, and proper HTTP status codes. Suggest a clear data model structure (employee id, name, role, salary). Add comments/docstrings for each endpoint. Provide complete working code in one file and basic API documentation. Also ensure secure and structured data handling.

Output:




http://localhost:3000/employees
GET
http://localhost:3000/employees
arams
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"name": "Siri",
"role": "Developer",
"department": "IT",
"salary": 50000
ody
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
JSON
[
{
"id": "39aa5c0c-9fd0-48c3-a233-a0383f088b28",
"name": "Siri",
"role": "Developer",


http://localhost:3000/employees/39aa5c0c-9fd0-48c3-a233-a0383f088b28/salary
PUT
http://localhost:3000/employees/39aa5c0c-9fd0-48c3-a233-a0383f088b28/salary
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"salary": 70000
}
Body
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
JSON
{
"message": "Salary updated successfully",
"employee": {
"id": "39aa5c0c-9fd0-48c3-a233-a0383f088b28",
"name": "Siri",



Explanation : The Employee Payroll API built using Flask manages employee data through REST endpoints.
It allows users to list employees, add new employees, update employee salary, and delete employees using GET, POST, PUT, and DELETE methods. Employee information is stored in an in-memory list and returned in JSON format.

Task 3: Student Information REST API

Prompt:

Create a Student Management REST API using Node.js and Express.

Features:

- Add student (name, rollNumber, course, year)

- Get all students

- Get student by ID

- Update student

- Delete student

Use:

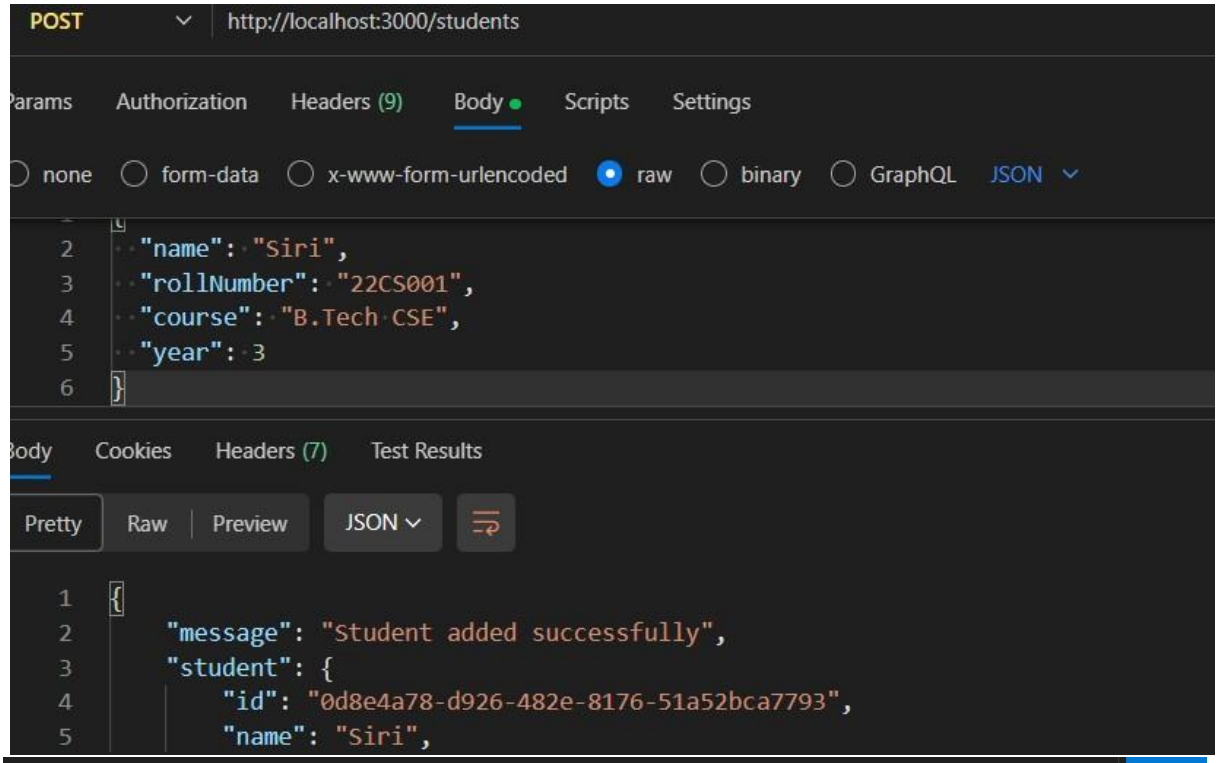
- In-memory storage

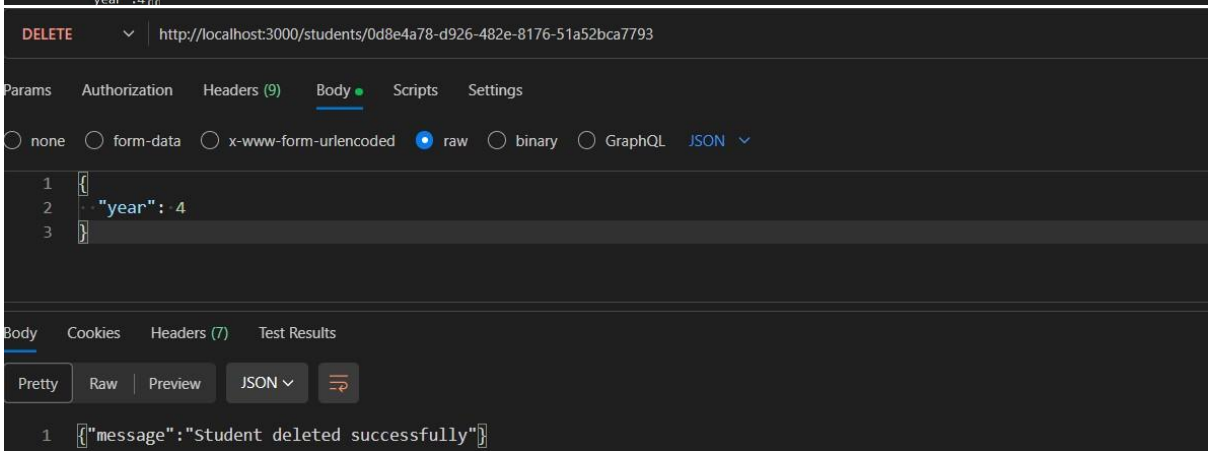
- UUID for IDs

- Input validation - Proper status codes

Output:

- Complete server.js with comments **Output:**





Explanation : This code is a simple API for a Student Information System built using the Flask framework. It allows users to perform CRUD (Create, Read, Update, Delete) operations on student data. The API has endpoints to list all students, add a new student, update an existing student's information, and delete a student based on their ID. The student data is stored in a list called `students`.

Task 4: Weather Information API Integration

Prompt:

Create a Weather API using Node.js and Express.

Endpoints:

- GET /weather/current/:city → current weather

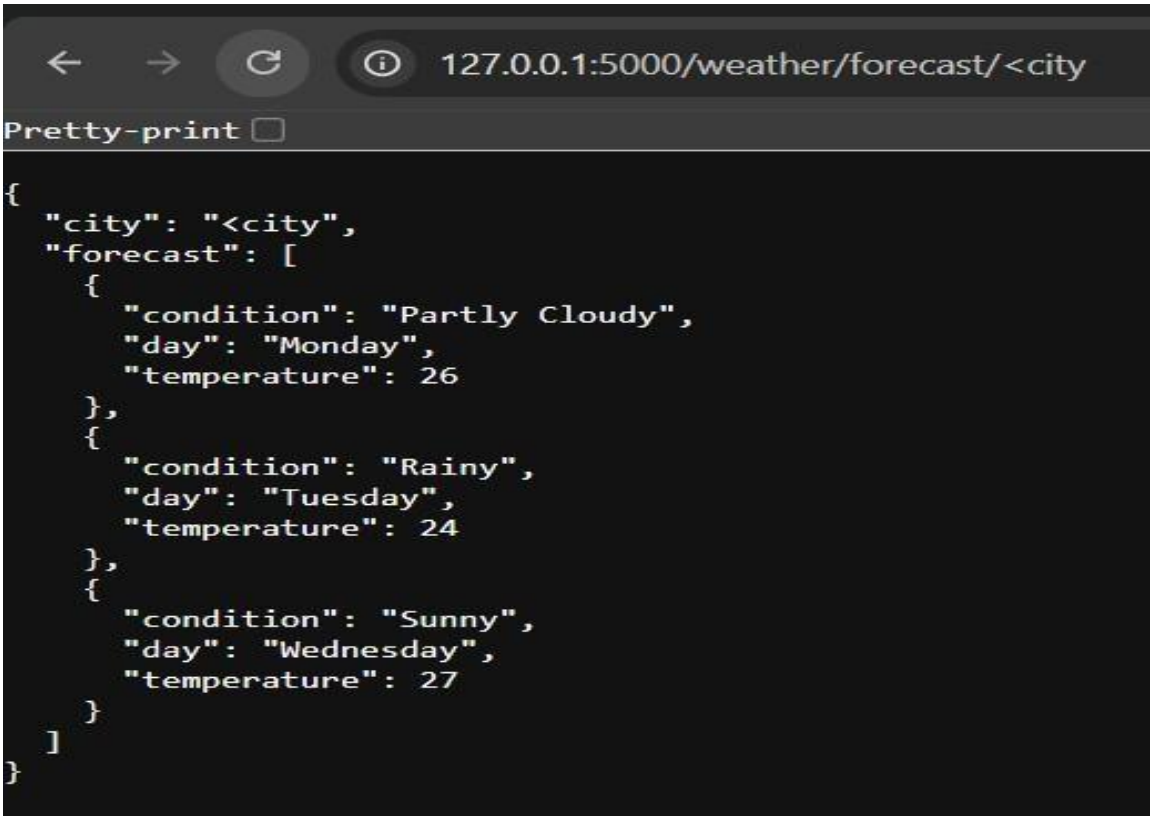
- GET /weather/forecast/:city → 5-day forecast

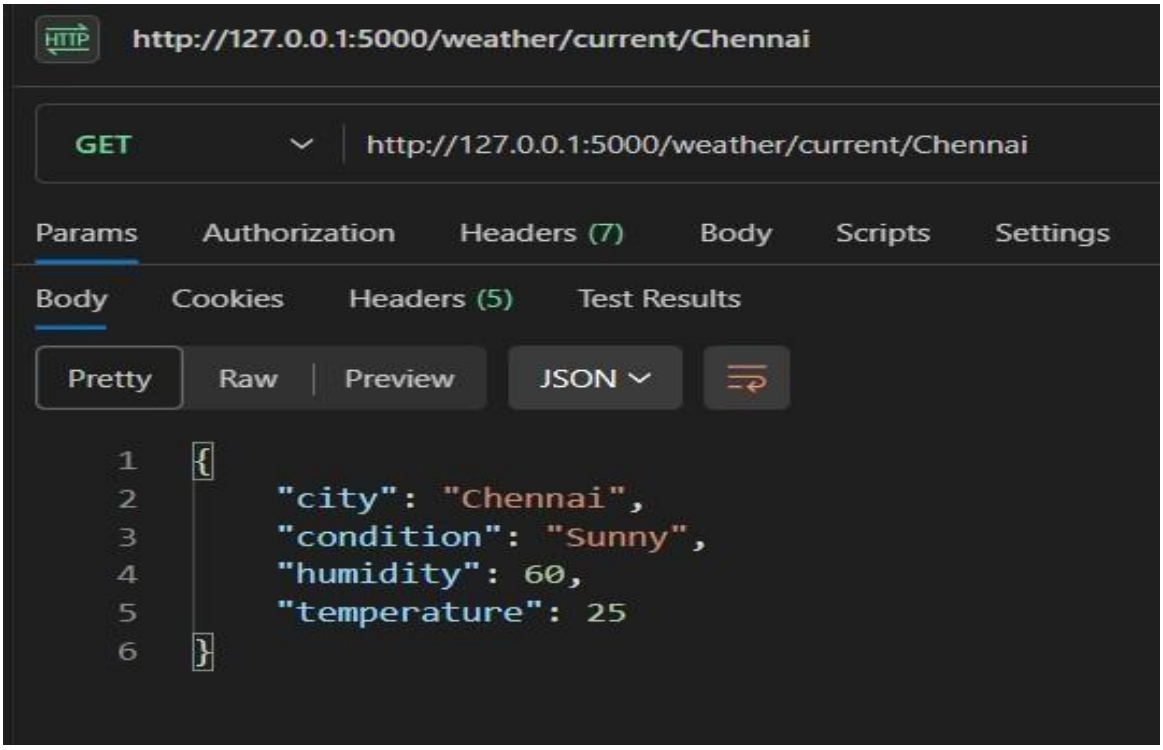
Use OpenWeatherMap API with axios.

Return temperature, humidity, and weather condition in JSON.

Include error handling and clean code.

Output:



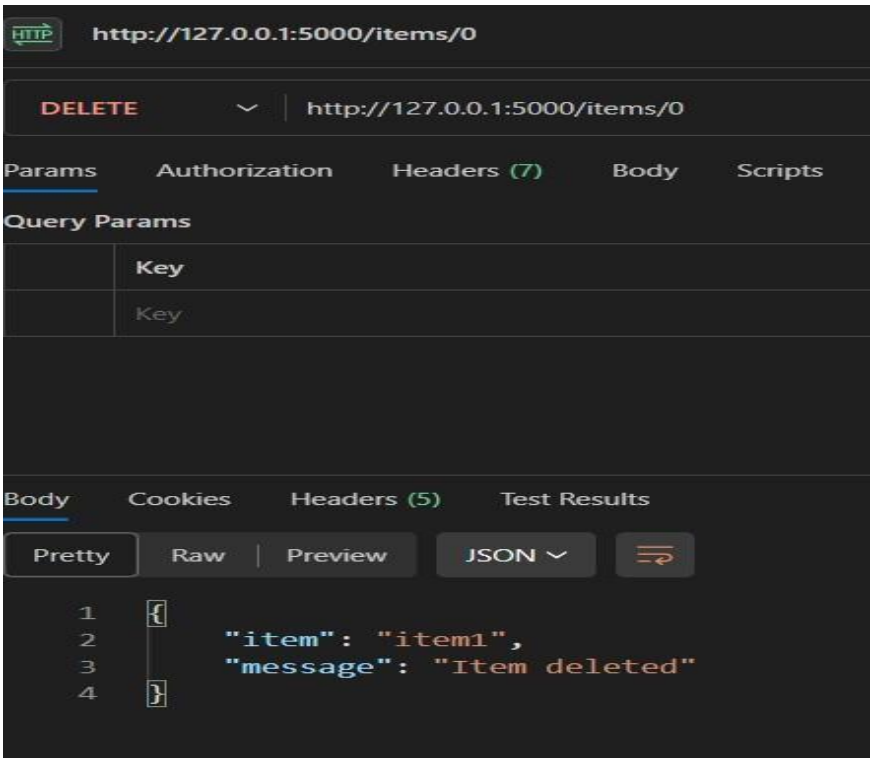


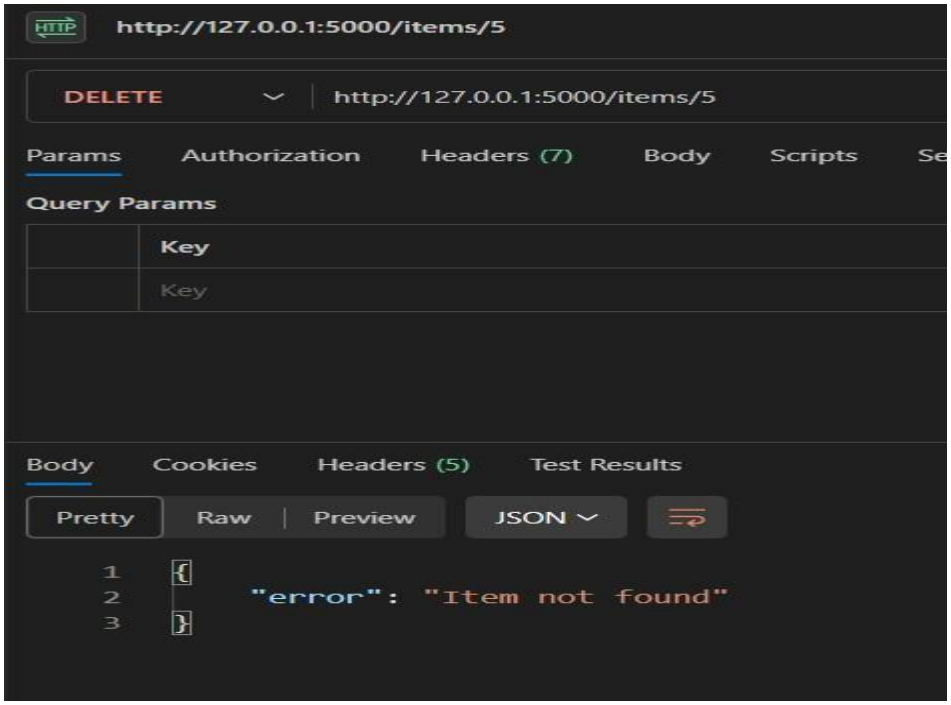
Explanation : This code creates a simple Flask REST API with two endpoints: /weather/current/<city> and /weather/forecast/<city>. The first endpoint returns mock current weather data for the specified city, while the second endpoint returns a mock weather forecast for the next few days. In a real implementation, you would replace the mock data with actual data fetched from the OpenWeatherMap API.

TASK – 05

Prompt : Create a Python **Flask REST API endpoint to delete an item from a list using the DELETE method. The endpoint should be /items/<int:index>. If the index is invalid, return a JSON error message "Item not found" with status code 404. If successful, remove the item and return a JSON response with "message": "Item deleted" and the deleted item.

Output:





Explanation :

This code creates a simple Flask REST API with an endpoint to delete an item from a list. The endpoint is defined as `/items/<int:index>`, where `<int:index>` is a placeholder for the index of the item to be deleted.

When a DELETE request is made to this endpoint, the function `delete_item` is called with the index as an argument. The function checks if the index is valid (i.e., within the bounds of the list). If it is valid, it removes the item from the list using `pop()` and returns a JSON response indicating that the item was deleted, along with the deleted item itself.